

test_scenario_sets_kinds.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/test_scenario_sets_kinds.vr

```
//
// Demonstrate how to use sets of scenario kinds
// i.e. groups of scenario defined as a different kinds of
// the same descriptor.
//
#include "scenarios.vrh"

//
// Telegraph good & bad cells
// good - BAD - good - BAD - good
// VPIa   VPIb  VPIa   VPIb  VPIa
//
class bad_cells extends atm_cell_scenario {
    integer TELEGRAPH;
    integer MANY_BAD;

    constraint telegraph_scenario {
        if (this.kind == this.TELEGRAPH) {
            length == 5;
            repeated == 0;
            foreach (this.items, i) {
                if (i % 2) this.items[i].hec != 8'h00;
                else      this.items[i].hec == 8'h00;
                if (i > 1) this.items[i].vpi == this.items[i-2].vpi;
                if (i == 1) this.items[i].vpi != this.items[i-1].vpi;
            }
        }
    }
}

constraint many_bad_scenario {
    if (this.kind == this.MANY_BAD) {
        length == 1;
        repeated in {3:5};
        foreach (this.items, i) {
            this.items[i].hec != 8'h00;
        }
    }
}
```

```

task new() {
    super.new();
    this.TELEGRAPH = this.define_scenario("Telegraph bad cells", 5);
    this.MANY_BAD = this.define_scenario("Many bad cells", 5);
}

task pre_randomize() {
    super.pre_randomize();
    foreach (this.items, i) {
        void = this.items[i].constraint_mode(OFF, "hec_valid");
    }
}

}

//
// Procedural scenario
//
class procedural_cells extends atm_cell_scenario {
    integer PROCEDURAL;

    constraint procedural_scenario {
        length == 0;
        repeated == 0;
    }

    task new() {
        super.new();
        this.PROCEDURAL = define_scenario("Procedural cells", 0);
    }

    virtual function integer apply_t(atm_cell_channel channel) {
        if (this.kind == this.PROCEDURAL) {
            atm_cell cell = new;
            void = cell.randomize() with {
                gfc == 4'h0;
                vpi == 8'h00;
                vci == 16'h0000;
                pt == 3'h0;
                clp == 1'b0;
            };
            repeat (3) {
                atm_cell c;
                cast_assign(c, cell.copy());
                channel.put_t(c);
            }
        }
    }
}

```

```

        void = cell.randomize() with {
            gfc == 4'hF;
            vpi == 8'hFF;
            vci == 16'hFFFF;
            pt  == 3'h7;
            clp == 1'b1;
        };
        channel.put_t(cell);
        apply_t = 4;
        return;
    }

    apply_t = super.apply_t(channel);
}
}

class my_atm_cell_scenario_gen_callbacks extends atm_cell_scenario_gen_callbacks{
    virtual task post_scenario_gen_t(atm_cell_scenario_gen gen,
                                     atm_cell_scenario scenario,
                                     var bit drop)
    {
        scenario.display();
        printf("-----\n");
    }
}

program test_gen {
    atm_cell_scenario_gen gen = new("Singleton");
    my_atm_cell_scenario_gen_callbacks cb = new;
    bad_cells s1 = new;
    procedural_cells s2 = new;
    gen.scenario_set.push_back(s1);
    gen.scenario_set.push_back(s2);
    gen.register_callback(cb);
    // Drain the output channel
    fork
    {
        while (1) {
            atm_cell cell = gen.out_chan.get_t();
        }
    }
    join none
}

```

```
    rvm_note(gen.log, "Generating 25 cells or 10 scenarios...");
    gen.stop_after_n_insts = 25;
    gen.stop_after_n_scenarios = 10;
    gen.start_xactor();
    void = gen.notify.wait_for_t(gen.DONE);
    delay(100);
    printf("=====\n");
    // Rnandomly pick scenarios from set now...
    rvm_note(gen.log, "Generating sets in random order...");
    void = gen.select_scenario.constraint_mode(OFF, "round_robin");
    gen.reset_xactor();
    gen.start_xactor();
    void = gen.notify.wait_for_t(gen.DONE);
    delay(100);
    gen.log.report();
}
```